

Javascript

Regular EXpressions

Rostagni Csaba

2026. április 27.

Tartalom

- 1 Regular Expressions
- 2 Regular Expressions JS kódban

Tartalom

1 Regular Expressions

- Bevezető
- Karakter meghatározása halmazként
- Számosság
- Speciális karakterek
- Csoportosítás
- Karakter osztályok

Regular Expressions

- RegEx
- RegExp
- Szabályos kifejezések
- Mintaillesztés
- Az "abc" lehet egy minta, ami ennyit jelent, hogy az abc karakterek egymás után jelenjenek meg.
- Összetettebb kifejezések írhatóak különböző behelyettesítésekkel

Tartalom

- 1 Regular Expressions
 - Bevezető
 - Karakter meghatározása halmazként
 - Számosság
 - Speciális karakterek
 - Csoportosítás
 - Karakter osztályok

Karakter megadása halmaz segítségével

- Halmazok jelölése: szögletes zárójel
- Pontosán 1 karaktert határoz meg
- pl.:
 - [a]
 - [A]
 - [abc]
 - [A-Z]

Karakter halmazok (intervallum)

Meghatározás intervallum segítségével

- A tól-ig értékeket kötőjellel jelölendőek
- `[0-9]` tetszőleges számjegy
- `[a-z]` kisbetű
- `[A-Z]` nagybetű
- `[a-zA-Z]` kis- vagy nagybetű

Figyelem!

Az ékezetes karakterek nem tartoznak ebbe bele, csak az **angol** ABC karakterei.

Karakter halmazok (felsorolás)

- A karaktereket közvetlen egymás után kell írni
- `[aA]` kicsi vagy nagy A betű
- `[ABCDEFGF]` a felsorolt karakterek egyike
- `[aáééííoóöőuúüű]` magánhangzók
- `[.]` egy pont karakter

Figyelem!

A karakterek felsorolásával ékezetes betűket is meg lehet adni, ha az adott környezet támogatja.

Figyelem!

A pont karakternek speciális jelentése van, ami halmazon belül nem érvényesül.

Karakter halmazok (vegyes)

- `[a-cD-F3-5]` Az (a,b,c,D,E,F,3,4,5) karakterek egyike
- `[A-CQT]` Az (A,B,C,Q,T) karakterek egyike

Figyelem!

A karakterek felsorolásával ékezetes betűket is meg lehet adni, ha az adott környezet támogatja.

Kivétel halmaz

- A kalap karakter (^) a tömb elején állva kizárja a halmaz elemeit
- `[^ABC]` bármi, ami se nem A, se nem B, se nem C karakter
- `[^A-Z]` bármi, ami nem része az angol abc nagybetűinek
- `[^0-9]` nem számjegy
- `[^.]` nem egy pont

Figyelem!

A kalap karakternek (^) több jelentése is van!

Tartalom

1 Regular Expressions

- Bevezető
- Karakter meghatározása halmazként
- Számosság
- Speciális karakterek
- Csoportosítás
- Karakter osztályok

Quantifiers - Mennyiségek meghatározása

- `*` Tetszőleges számú (0, 1, 2, ..., n) ismétlődés
- `+` Legalább egyszeri ismétlődés
- `?` Vagy van, vagy nincs. (ismétlődések száma 0 vagy 1)

Linkek:

- [Quantifiers \(MDN\)](#)
- [CS50 Lecture 7 2022 \(youtube - Harvard\)](#)

Quantifiers - Mennyiségek meghatározása

- $\{n\}$ Pontosán n -szer ismétlődik, ahol
 - n egy pozitív egész szám
- $\{n,m\}$ Legalább n -szer, legfeljebb m -szer ismétlődik, ahol
 - n pozitív egész szám vagy 0
 - $n < m$
- $\{n,\}$ Legalább n -szer ismétlődik, ahol
 - n pozitív egész szám

A kapcsos zárójelek a részét képezik a kifejezéseknek!

Tartalom

1 Regular Expressions

- Bevezető
- Karakter meghatározása halmazként
- Számosság
- **Speciális karakterek**
- Csoportosítás
- Karakter osztályok

Speciális karakterek

- `\` semlegesíti a speciális jelentését a következő karakternek!
- `\t` tabulátor
- `\r` kocs vissza
- `\n` soremelés
- `\0` NULL

Windows alatt `\r\n` a sortörés, míg a legtöbb Linux és Mac rendszeren a `\n`, ugyanakkor régebbi Mac rendszereken pont a `\r` volt.

A pont

- `.` A pont karakter egyetlen tetszőleges karaktert helyettesít
- `\.` A backslash hatására elveszti speciális jelentését, és ténylegesen egy pontot jelent
- `[.]` Ha tömbként van meghatározva, szintén egy sima pontot jelent.

Figyelem!

Az újsor karakterre nem illeszkedik!

Elejés és vége

- A minta egy szövegen belül tetszőleges helyen megtalálható
- A kalap karakter (^) a szöveg **elejét** jelzi.
- A dollár karakter (\$) a szöveg **végét** jelzi.

Figyelem!

A kalap jel karakterhalmazok esetén más jelentéssel bír!

Info

A kalap karakter (^) windows rendszeren `alt gr + 3` billentyűkombinációval csálható elő. Ugyanakkor elsőre nem írja ki, másodjára pedig megduplázza. Mac esetében `option + á`

Tartalom

1 Regular Expressions

- Bevezető
- Karakter meghatározása halmazként
- Számosság
- Speciális karakterek
- Csoportosítás
- Karakter osztályok

Csoportosítás

- `()` zárójelekkel csoportokat alakíthatunk ki
- `|` vagy
- `(?:)` non-capturing group nem csinál csoportot az illeszkedésre
- `(?<nev>)` named capturing group
- `(ok){2}` Az `ok` karaktersorozat ismétlődjön kétszer
- `(yes|no)` Csak a `yes` és a `no` az elfogadott karaktersorozat
- `(.*)((?:[@])(.))` Csak az érdekes, hogy mi található a kukac előtt és utána. DE a kettő között egy kukac áll!
- `(?<ev>[0-9]{4})-(?<ho>[0-9]{2})-(?<nap>[0-9]{2})` Az `ev`, `ho`, `nap` későbbiekben hivatkozható

Tartalom

1 Regular Expressions

- Bevezető
- Karakter meghatározása halmazként
- Számosság
- Speciális karakterek
- Csoportosítás
- Karakter osztályok

Karakter osztályok

- `\d` digit számjegy `[0-9]`
- `\D` digit NEM számjegy `[^0-9]`
- `\w` alfanumerikus karakter és az aláhúzás `[a-zA-Z0-9_]`
- `\W` se nem alfanumerikus karakter, se nem aláhúzás
- `\s` Egy whitespace karakter (szóköz, tabulátor, sortörés, stb)
- `\S` Nem whitespace karakter

Linkek:

- [Character classes \(MDN\)](#)

Karakter osztályok

A megadott Unicode karakterosztályban lévő elemekre szűr

- `\p{Emoji_Presentation}` emoji karaktert tartalmaz
- `\p{Lower}` kisbetű
- `\p{Upper}` nagybetű
- `\p{White_Space}` szóköz, tabulátor, sortörés
- `\w` alfanumerikus karakter és az aláhúzás `[a-zA-Z0-9_]`
- `\W` se nem alfanumerikus karakter, se nem aláhúzás
- `\s` Egy whitespace karakter (szóköz, tabulátor, sortörés, stb)
- `\S` Nem whitespace karakter

Linkek:

- [Character classes \(MDN\)](#)
- [Binary Unicode property aliases](#)

Tartalom

- 1 Regular Expressions
- 2 Regular Expressions JS kódban

Tartalom

2 Regular Expressions JS kódban

- Bevezető
- Metódusok

RegExp (JS)

- JavaScriptben két féle képpen lehet létrehozni reguláris kifejezést
 - regular expression literal segítségével
 - A RegExp osztály konstruktorával
- Ismert, fix regex esetén előbbi
- Ismeretlen, külső változóból származó regex esetén utóbbit célszerű alkalmazni

```
const re = /[1-9][0-9]* Ft/gi;
```

regular expression literal

```
const re = new RegExp('[1-9][0-9]* Ft', 'gi');
```

RegExp konstruktor

Linkek:

- [RegExp \(MDN\)](#)

Tartalom

2 Regular Expressions JS kódban

- Bevezető
- Metódusok

test()

- `test(szoveg)` A szövegen megkeresi az adott mintát
 - igaz, ha a minta megtalálható a szövegben
 - különben hamis

```
const s1 = "András"
```

```
const s2 = "and"
```

```
const regex = /An.*/
```

```
console.log(regex.test(s1)) // true
```

```
console.log(regex.test(s2)) // false
```

JS

- Mivel a `i` kapcsoló nem szerepel a végén, így a kis- és nagybetűket megkülönbözteti

exec()

- `exec(szoveg)` Végrehajtja az illesztést a szövegen
 - ha van eredmény objektumként visszaadja
 - különben null
 - az egyes csoportokat indexelhetőek

JS

```
const html1 = `

<b>alma</b> <i>brack</i><b>citrom</b></p>`
const regex1 = /<b>([^\<]*)<\b>/
const regex2 = /<b>([^\<]*)<\b>/g
regex1.exec(html1) // ['<b>alma</b>', 'alma']
regex2.exec(html1) // ['<b>alma</b>', '<b>citrom</b>']
const html2 = `

<u>alma</u> <i>brack</i><i>citrom</i></p>`
regex1.exec(html2) // null
regex2.exec(html2) // null


```

match()

- `match(regex)` A szövegen megkeresi az adott mintát
 - ha van eredmény tömbként visszaadja
 - különben null

JS

```
const napok = "Hétfő, Kedd, Szerda"
const regex1 = /[A-Z][a-z]/g
const regex2 = /\p{Upper}\p{Lower}/gu

eredmeny = napok.match(regex1) // ['Ke', 'Sz']
eredmeny = napok.match(regex2) // ['Hé', 'Ke', 'Sz']
```

- Az `[a-z]` halmazban nincsenek benne a magyar ékezetes betűk

matchAll()

- `match(regex)` A szövegen megkeresi az adott mintát
 - ha van eredmény tömböket tartalmazó iterátorként adja vissza
 - különben null
 - A global flag legyen bekapcsolva!

JS

```
const napok = "Hétfő, Kedd, Szerda"
```

```
const regex1 = /[A-Z][a-z]/g
```

```
eredmeny = [... napok.matchAll(regex1)];
```

```
console.log(e[0]) // ['Ke', index: 7, input: 'Hétfő, Kedd,  
↪ Szerda', groups: undefined]
```

search

- `search(regex)` A szövegen megkeresi az adott mintát
 - ha az illesztés sikeres az indexét adja vissza
 - különben -1

JS

```
let str = "hey Jude"  
let re = /[A-Z]/g  
let reDot = /[.]/g
```

```
console.log(str.search(re))  
// Az első nagybetű indexe 4 lesz.
```

```
console.log(str.search(reDot))  
// A "." nem található meg a szövegben, így a visszaadott  
↪ érték -1 lesz
```